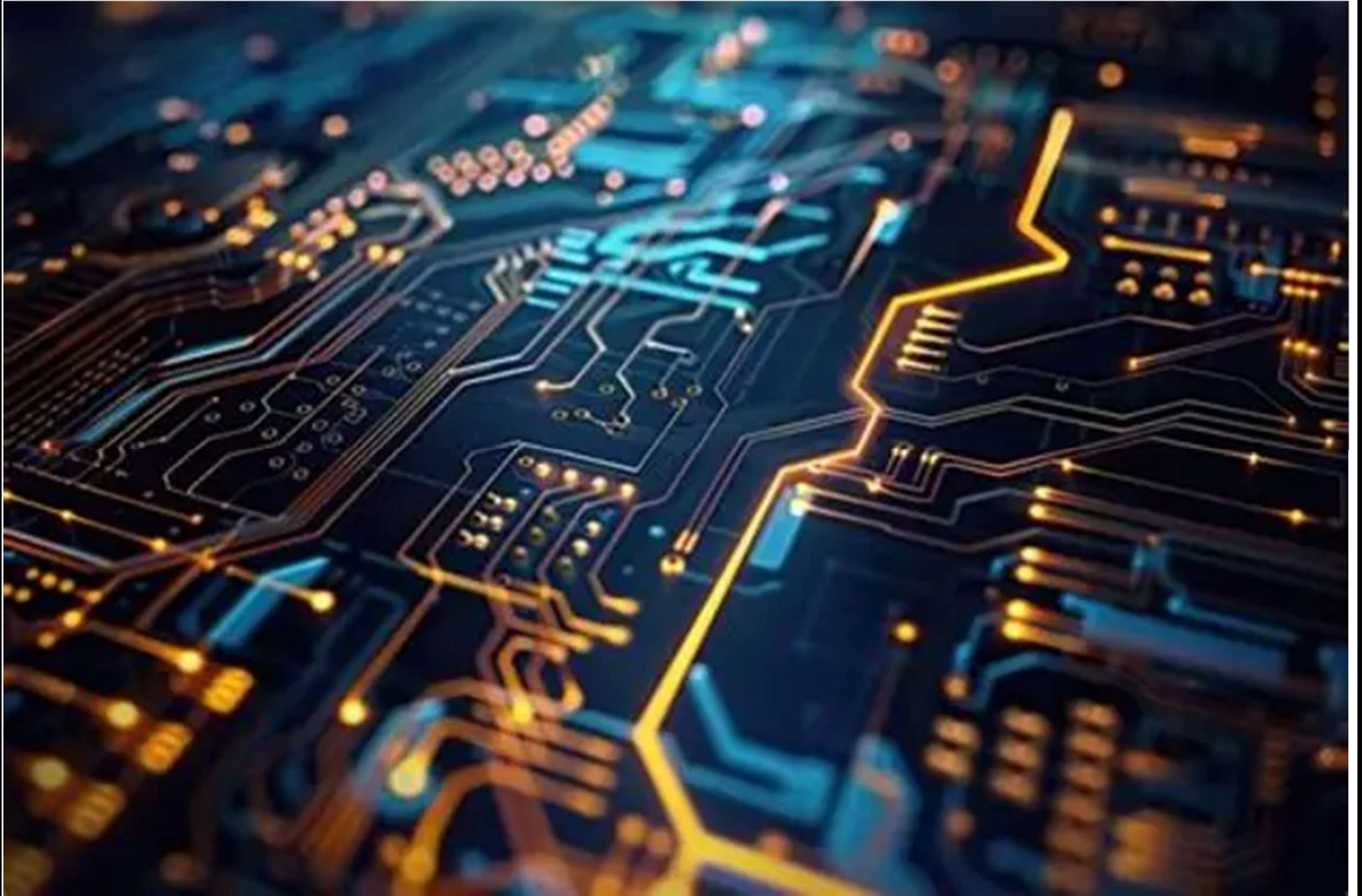


Lab: Generic Decoder and 4*2 Encoder



Name: Ahmed Wael Sadek El
Sawy

Lab: Generic Decoder and 4*2 Encoder

Generic Decoder

generic_decoder.v

```
module generic_decoder #(
    parameter N = 3 // N inputs, 2^N outputs
)(
    input  wire          en,
    input  wire [N-1:0]  in,
    output reg [(1<<N)-1:0] out
);

    always @(*) begin
        if (en) begin
            out = 1 << in;
        end else begin
            out = 0;
        end
    end
endmodule
```

tb_generic_decoder.v

```
module tb_generic_decoder;
    parameter N = 3;
    parameter OUT_WIDTH = 1 << N;

    // Testbench signals
    reg          en;
    reg [N-1:0]  in;
    wire [OUT_WIDTH-1:0] out;

    reg [OUT_WIDTH-1:0] expected_out;
    integer errors = 0;
    integer i;

    generic_decoder #(.N(N)) DUT (
        .en(en),
        .in(in),
        .out(out)
    );

    initial begin
        // Initialize
        en = 0; in = 0;
    end
endmodule
```

Lab: Generic Decoder and 4*2 Encoder

```
// Test 1: Disabled State
#10;
if (out !== 0) begin
    $display("ERROR: Decoder not 0 when disabled (en=0). Got: %b", out);
    errors = errors + 1;
end else begin
    $display("PASS: Disabled State -> Output %b", out);
end

// Test 2: Iterate through all possible inputs
en = 1;
for (i = 0; i < OUT_WIDTH; i = i + 1) begin
    in = i;
    expected_out = 1 << i;

    #10;

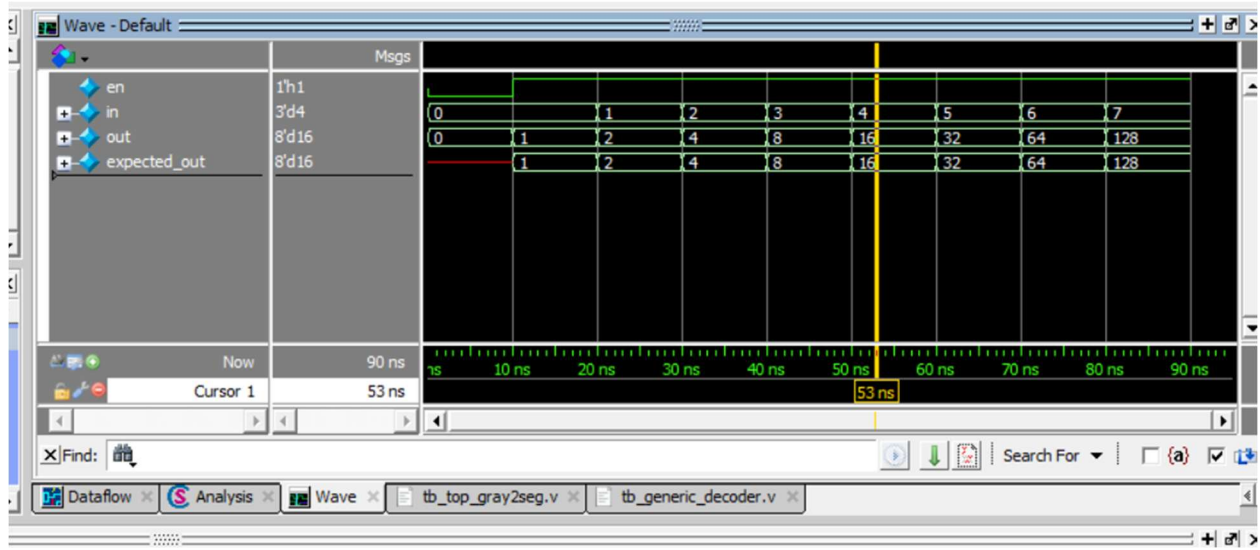
    if (out !== expected_out) begin
        $display("ERROR: For input %d, expected %b, got %b", in,
expected_out, out);
        errors = errors + 1;
    end else begin
        $display("PASS: Input %d -> Output %b", in, out);
    end
end

if (errors == 0) $display("SUCCESS: Parameterized Decoder passed all
tests.");
else $display("FAILED with %0d errors.", errors);
$stop;
end
endmodule
```

Testing generic decoder:

Lab: Generic Decoder and 4*2 Encoder

```
# Loading work.generic_decoder(fast)
# PASS: Disabled State -> Output 00000000
# PASS: Input 0 -> Output 00000001
# PASS: Input 1 -> Output 00000010
# PASS: Input 2 -> Output 00000100
# PASS: Input 3 -> Output 00001000
# PASS: Input 4 -> Output 00010000
# PASS: Input 5 -> Output 00100000
# PASS: Input 6 -> Output 01000000
# PASS: Input 7 -> Output 10000000
# SUCCESS: Parameterized Decoder passed all tests.
# ** Note: $stop      : tb_generic_decoder.v(51)
# Time: 90 ns Iteration: 0 Instance: /tb_generic_decoder
```



4*2 Encoder

encoder_4x2.v

```
module encoder_4x2 (
    input wire en,
    input wire [3:0] in,
    output reg [1:0] out
);
    always @(*) begin
        if (en) begin
            casez (in)
                4'b1??? : out = 2'd3;
                4'b01?? : out = 2'd2;
                4'b001? : out = 2'd1;
                4'b0001 : out = 2'd0;
                default : out = 2'd0;
            endcase
        end
    end
endmodule
```

Lab: Generic Decoder and 4*2 Encoder

```
        end else begin
            out = 2'd0;
        end
    end
endmodule
```

tb_encoder_4x2.v

```
module tb_encoder_4x2;
    // Testbench signals
    reg      en;
    reg [3:0] in;
    wire [1:0] out;

    integer errors = 0;

    encoder_4x2 DUT (
        .en(en),
        .in(in),
        .out(out)
    );

    initial begin
        // 1. Test Disabled State
        en = 0; in = 4'b1111;
        #10;

        if (out !== 2'd0) begin
            $display("ERROR: Encoder should output 0 when disabled (en=0). Got:
            %b", out);
            errors = errors + 1;
        end else begin
            $display("PASS: Disabled State -> out=%b", out);
        end

        // 2. Test Normal 1-Hot Encoding
        en = 1;

        in = 4'b0001; #10;
        if (out !== 2'd0) begin
            $display("ERROR for input %b, got %d", in, out); errors = errors + 1;
        end else $display("PASS: Input %b -> Output %d", in, out);

        in = 4'b0010; #10;
        if (out !== 2'd1) begin
            $display("ERROR for input %b, got %d", in, out); errors = errors + 1;
        end
    end
endmodule
```

Lab: Generic Decoder and 4*2 Encoder

```
end else $display("PASS: Input %b -> Output %d", in, out);

in = 4'b0100; #10;
if (out !== 2'd2) begin
    $display("ERROR for input %b, got %d", in, out); errors = errors + 1;
end else $display("PASS: Input %b -> Output %d", in, out);

in = 4'b1000; #10;
if (out !== 2'd3) begin
    $display("ERROR for input %b, got %d", in, out); errors = errors + 1;
end else $display("PASS: Input %b -> Output %d", in, out);

// 3. Test Priority Logic (Multiple 1s)
in = 4'b1101; #10; // MSB is 1, should prioritize to output 3
if (out !== 2'd3) begin
    $display("ERROR for Priority Input %b, expected 3, got %d", in, out);
errors = errors + 1;
end else begin
    $display("PASS: Priority Input %b -> Output %d", in, out);
end

in = 4'b0111; #10; // Bit 2 is highest, should prioritize to output 2
if (out !== 2'd2) begin
    $display("ERROR for Priority Input %b, expected 2, got %d", in, out);
errors = errors + 1;
end else begin
    $display("PASS: Priority Input %b -> Output %d", in, out);
end

if (errors == 0) $display("SUCCESS: Normal 4x2 Encoder passed all
tests.");
else $display("FAILED with %0d errors.", errors);

$stop;
end
endmodule
```

Testing 4*2 Encoder:

Lab: Generic Decoder and 4*2 Encoder

```
# Loading work_encoder_4x2 (1850)
# PASS: Disabled State -> out=00
# PASS: Input 0001 -> Output 0
# PASS: Input 0010 -> Output 1
# PASS: Input 0100 -> Output 2
# PASS: Input 1000 -> Output 3
# PASS: Priority Input 1101 -> Output 3
# PASS: Priority Input 0111 -> Output 2
# SUCCESS: Normal 4x2 Encoder passed all tests.
# ** Note: $stop      : tb_encoder_4x2.v(68)
#   Time: 70 ns   Iteration: 0   Instance: /tb_encoder_4x2
# Break in Module tb_encoder_4x2 at tb_encoder_4x2.v line 68
```

